

NoteShare EFREI

Green IT project

I. Introduction

Students often spend a significant amount of time searching for a reliable and well structured course. They can struggle to make one themselves, or rely on scattered resources shared online, or through group chats. These solutions are however often disorganized and difficult to navigate properly.

As a result, students lose time and efficiency during their study sessions. There is therefore a need for a centralized platform that allows users to easily access, share and organize study resources.

Our project aims to provide a lightweight file sharing web platform, dedicated to education resources, specifically students own study sheets.

The platform allows students to:

- Access structured and categorized materials
- Share their own notes
- Interact with other users through comments

By organizing content by subject and providing a simple navigation system, the platform lowers the amount of time spent searching for information, thus improving the efficiency of study sessions.

The platform is intended for three main types of users:

- Students: the main users who upload, consult and interact with study materials
- Administrator: users responsible for managing the platform, moderating content, and ensuring compliance with platform rules and data policies.

The MVP scope focuses on the core functionalities required for a functional and useful platform:

- User authentication (sign up/ in)
- Uploading study sheets
- Viewing and downloading study sheets
- Basic content organization by subject
- Comment section under each study sheet

These features ensure that the platform delivers immediate value while remaining as simple as possible.

From the design phase, the project incorporates several digital sobriety principles:

- Use as less frontend technologies as possible to reduce client side resource consumption

- We selected an efficient backend language (Go) to ensure low memory and CPU usage
- Lazy loading of content (study sheets are only loaded when requested by the user)
- Optimized data storage and queries to avoid unnecessary computations
- Consideration of low-energy deployment environments (eg., Raspberry Pi)

II. Architecture and Design

The system is structured around a client–server architecture, where the client interface interacts with a backend server responsible for processing requests and communicating with the database.

The application is organized into several functional components:

- A navigation system allowing users to browse study sheets through a hierarchical structure based on categories and subcategories
- A study sheet module responsible for managing uploaded files and their associated metadata
- A comment system, enabling users to interact and exchange feedback on study sheets
- An authentication system, handling user login, account creation, and access control

When a user interacts with the platform, the client sends a request to the server which processes it and queries the database when necessary. The results are then returned to the client and displayed dynamically.

The data model is based on five main entities: Account, StudySheet, Comment, Category, and SubCategory.

- An Account represents a user of the platform and stores authentication and identification data
- A StudySheet represents a shared learning resource and is associated with a specific subcategory and user
- A Comment allows users to interact by posting feedback on study sheets
- A Category groups study sheets into broad domains (e.g., mathematics, physics)
- A SubCategory refines this classification into more specific topics

The relationships between these entities are defined as follows:

- One account can create multiple study sheets
- One study sheet belongs to a single subcategory
- One category can contain multiple subcategories
- One study sheet can receive multiple comments
- One account can post multiple comments

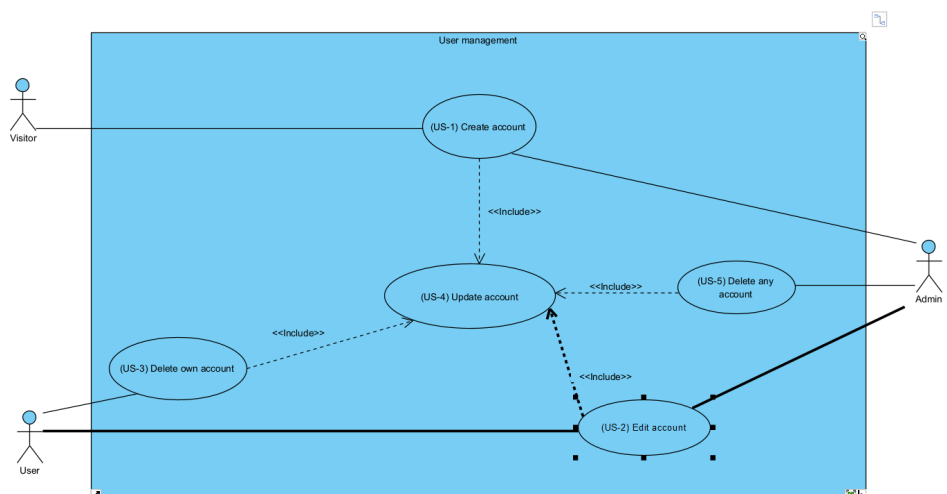
Like any other apps, ours will be divided into three main layers:

The frontend, which is implemented using HTML, CSS, and a minimal amount of JavaScript. It is responsible for rendering the user interface and handling user interactions. The backend is developed in Go and handles business logic, authentication, request processing, and communication with the database. And the database layer, implemented using MySQL, is responsible for storing and managing persistent data, including user accounts, study sheets, comments, and category structures.

User story:

- US-01: As a visitor, I want to create an account so that I can become a user.
- US-02: As a user, I want to edit my account information so that I can keep my personal data up to date.
- US-03: As a user, I want to delete my account so that I no longer appear in the system and my data can be removed.
- US-04: As the system, I want to update user accounts so that information remains consistent and platform rules are enforced.
- US-05: As an admin, I want to delete user accounts so that I can enforce rules and handle account deletion requests.
- US-06: As an admin, I want to create accounts so that I can manage users when necessary.
- US-07: As an admin, I want to edit any account so that I can correct or manage user information.
- US-08: As a user, I want to upload study sheets so that I can share learning resources.
- US-09: As a user, I want to view study sheets so that I can access learning materials.
- US-10: As a visitor, I want to download study sheets so that I can consult them offline.
- US-11: As a user, I want to comment on a study sheet so that I can ask questions or provide help.
- US-12: As a user, I want to request an update to a study sheet so that improved versions can replace outdated ones.
- US-13: As an external evaluator, I want to access data policies so that I can assess the platform's energy consumption.

hare Use case diagram]



Fig

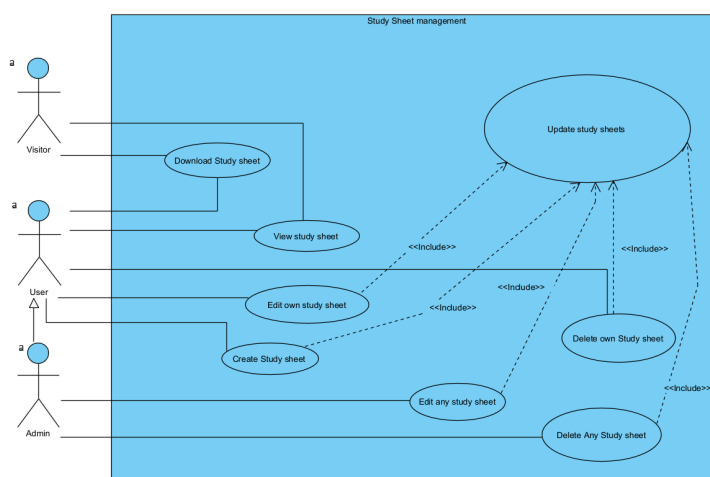


Fig 2-2

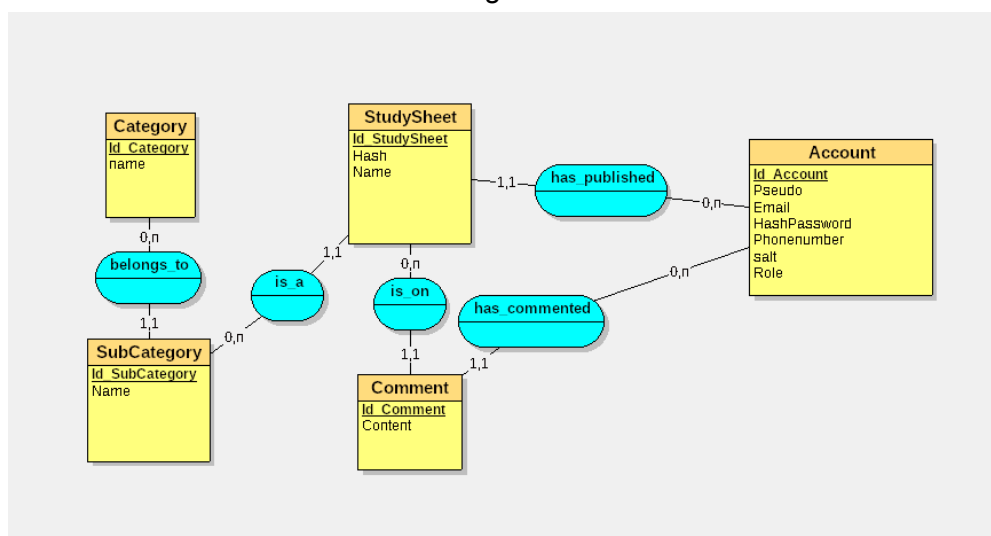


Fig 2-3

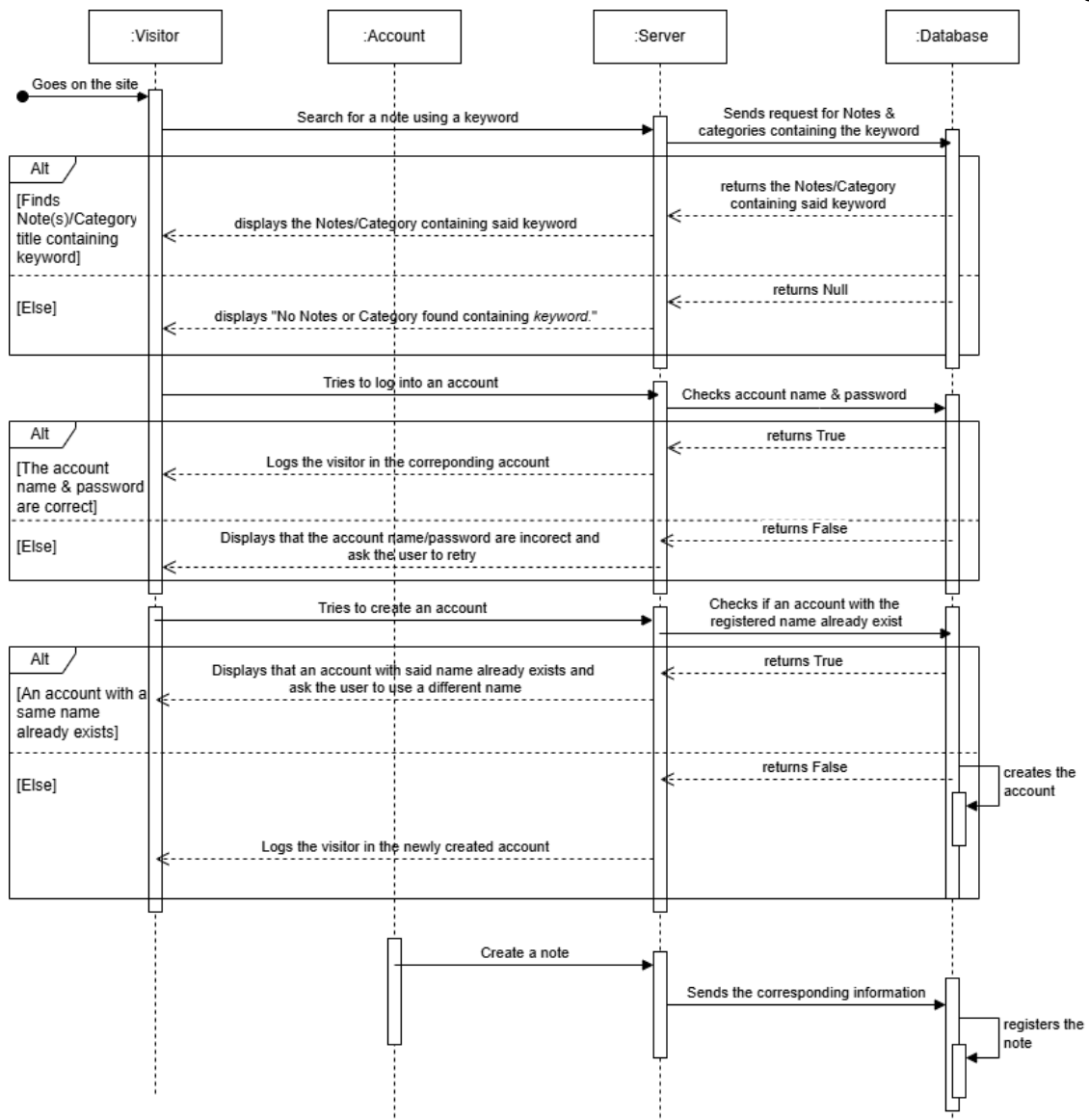
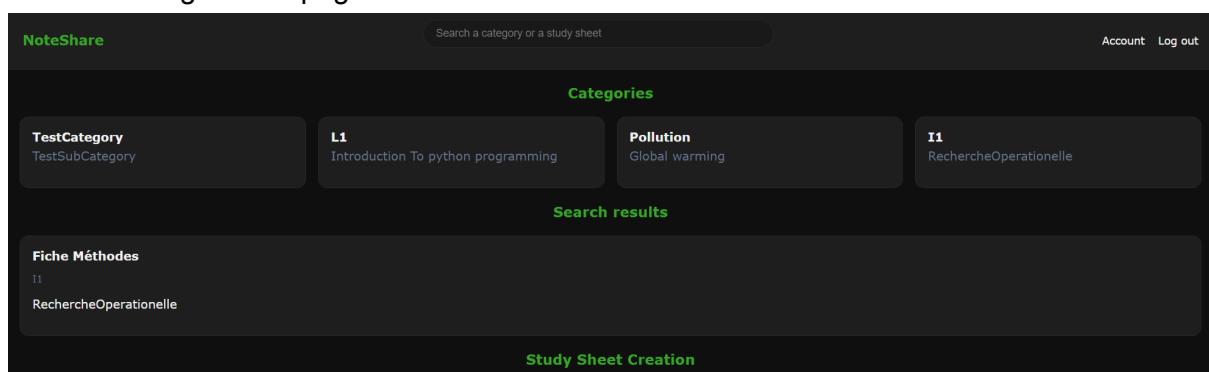


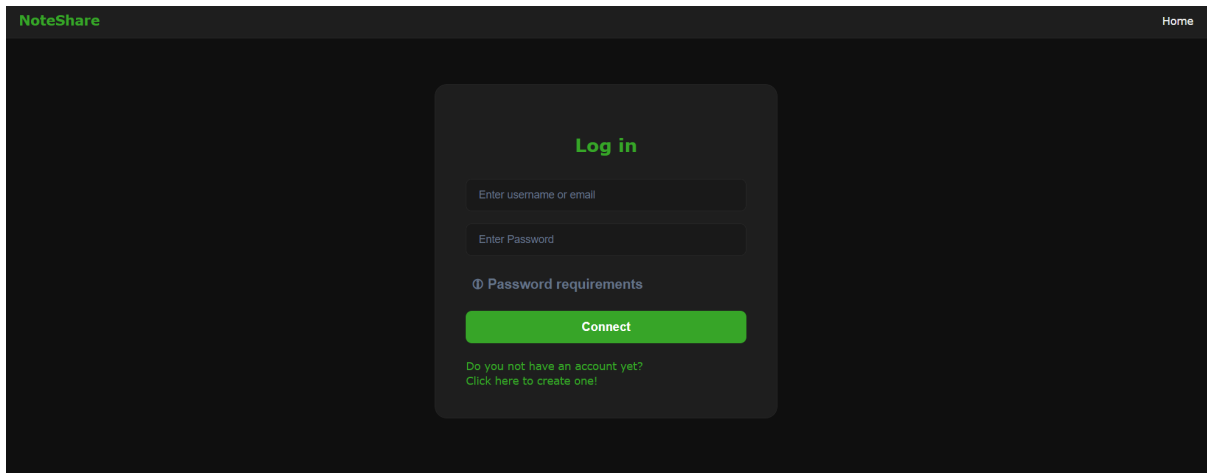
Fig 2-4

Interface design: Main page



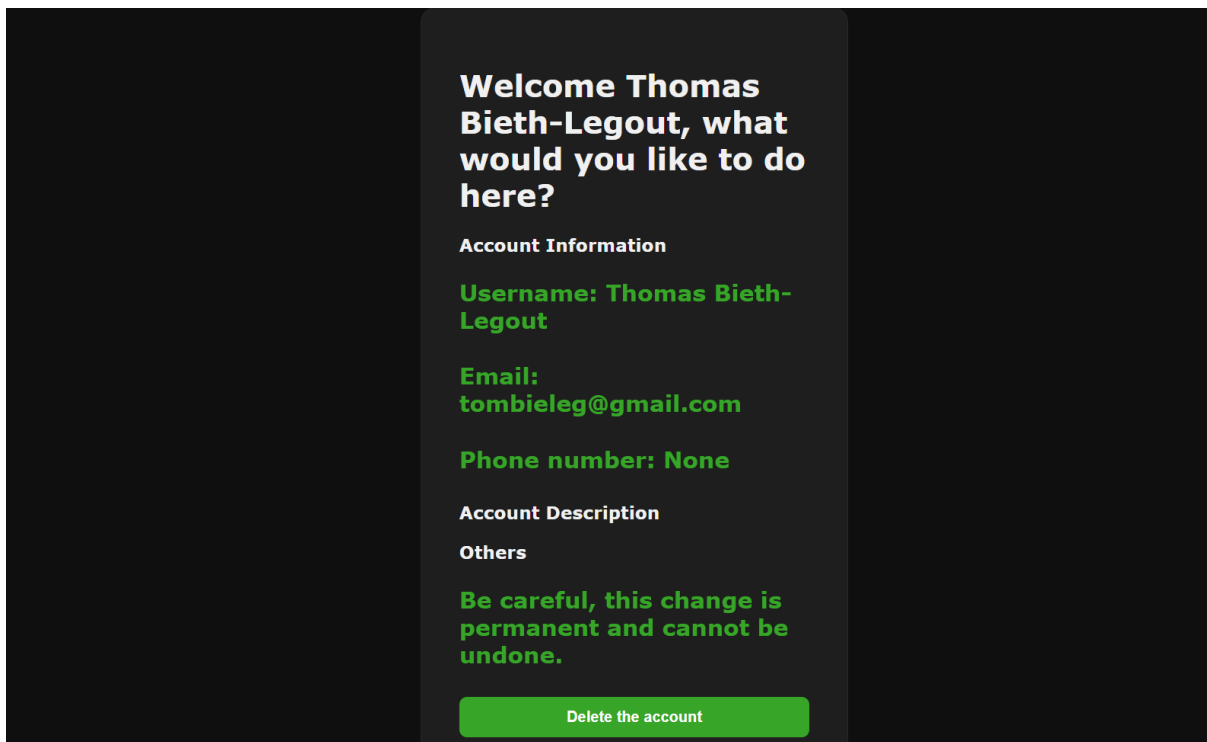
Alexis Delavis
Jude Aybalen
Thomas Bieth–Legout
Guillaume Bernard
Alexis Lafargue

Log in page:



The image shows a dark-themed login page for 'NoteShare'. At the top left is the 'NoteShare' logo and at the top right is a 'Home' link. The main content is a light gray rounded rectangle containing the title 'Log in' in green. Below the title are two input fields: 'Enter username or email' and 'Enter Password'. Under the password field is a link 'ⓘ Password requirements'. A green 'Connect' button is positioned below these fields. At the bottom of the rectangle, there is a green link that says 'Do you not have an account yet? Click here to create one!'.

Account page:



The image shows a dark-themed account page for 'NoteShare'. The main content is a light gray rounded rectangle. At the top, it says 'Welcome Thomas Bieth-Legout, what would you like to do here?'. Below this is the section 'Account Information' which lists: 'Username: Thomas Bieth-Legout', 'Email: tombieleg@gmail.com', and 'Phone number: None'. The next section is 'Account Description' followed by 'Others'. A green warning message states: 'Be careful, this change is permanent and cannot be undone.' At the bottom of the rectangle is a green button labeled 'Delete the account'.

III. Technology choices

The technologies used for this project were chosen with two main ideas in mind: keeping the application simple to build and maintain, while limiting unnecessary resource usage.

For the frontend, we used HTML, CSS and a small amount of JavaScript. Since the platform does not require complex interactions, using heavier frameworks would not bring much benefit. Keeping things simple helps reduce loading time and avoids adding unnecessary processing on the client side.

For the backend, we chose Go. It is well suited for building web services and allows us to handle requests efficiently without relying on many external tools. It also helped that the team was already somewhat familiar with it, which made development smoother.

The database is handled with MySQL. It provides everything needed for managing structured data in this project, without adding extra complexity.

For deployment, we opted for Docker, mainly to simplify setup and ensure the application runs in the same environment everywhere. The project is also designed to run on a low-power machine such as a Raspberry Pi, which fits with the goal of keeping resource usage low.

Finally, we used GitHub for version control and collaboration, with a simple branching workflow and pull requests to review changes. which is very useful when working in teams.

IV. Implementation

Link to our github : <https://github.com/NoteShareEFREI/NoteShareEFREI>

Implemented functionalities :

Authentication and account management:

- User registration with a strong password policy
 - Password length between 8 and 31 characters
 - Mandatory inclusion of all types of characters
 - Rejection of unsafe character to prevent injection attacks
- Authentication mechanism
 - Use of JWT
 - Secure cookie configuration
- Account Management
 - Deletion of accounts via API
 - Role-based access control (admin, user)

Standard CRUD system for study sheets

- Create

- Upload of pdf files
- automatic generation of sha-256 for file identification and deduplication
- Read
 - Listing of study sheets with filtering by category
 - Text search functionality
- Update
 - Not currently implemented as not fully necessary currently
- Delete
 - Deletion only through removing of associated subcategories or categories

Hierarchical category management

- Structure
 - Category (parent)
 - Subcategory (child)
- Operations
 - CRUD functionality
 - Cascading deletion using database transactions

Comment System

- Creation of comments associated with study sheets
- Retrieval of comments with author username (via JOIN with Account table)
- Authentication required for posting comments
- Chronological ordering of comments

Administrative role and data management panel

- Accessible via /admin
- Access control enforced via middleware (Role=Admin)
- Creation and deletion of categories and subcategories

Implemented optimizations:

Database indexing:

- Optimize lookup performance

Template caching:

- Avoids repeated parsing by loading templates into memory at startup

Prepared statements

- Enhances performance and prevents SQL injection

Connection pooling

- Efficient reuse of database connections

Stateless authentication

- eliminates server-side session storage

Centralized authentication middleware

- Reduces Code duplication

Transactional operations

- Guarantees data integrity

Technologies used:

Domain	Technologies
Backend	Go
Database	MySQL
Authentication	JWT
Hashing	BCrypt with salt
Frontend	HTML, CSS, JavaScript
Deployment	Docker, DockerCompose
Hosting	Raspberry pi

Main Data flows:

- Signup process:
Input -> Validation -> Hashing -> Database insert -> JWT -> Redirect
- Login process:
Input -> Verification -> JWT -> Redirect
- Study sheet upload:
Authentication -> File processing -> Hashing -> Storage -> Database insert
- Search process:
Request -> Query execution -> Data processing -> Template rendering
- Comment system:
Retrieval via API -> Database query -> JSON Response
Submission via API -> Authentication -> Insert -> Response

V. Carbon Footprint Analysis

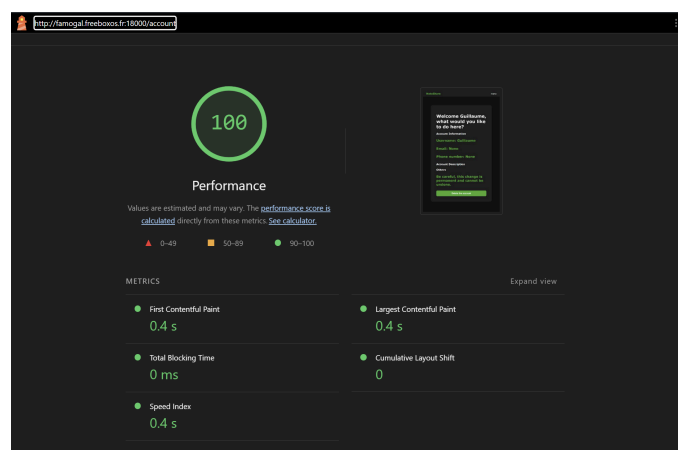
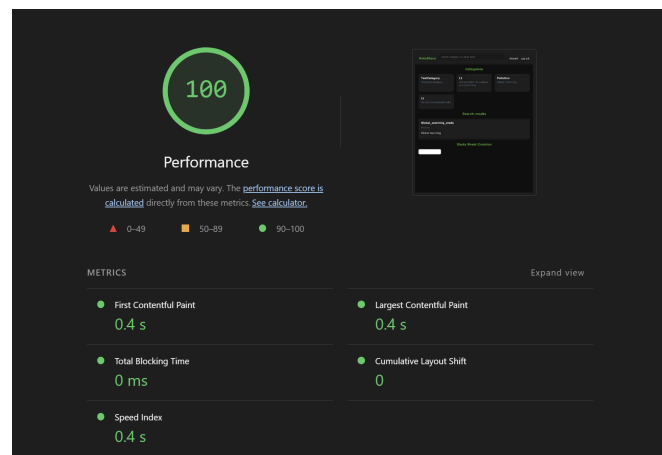
Comparison of our website with Moodle

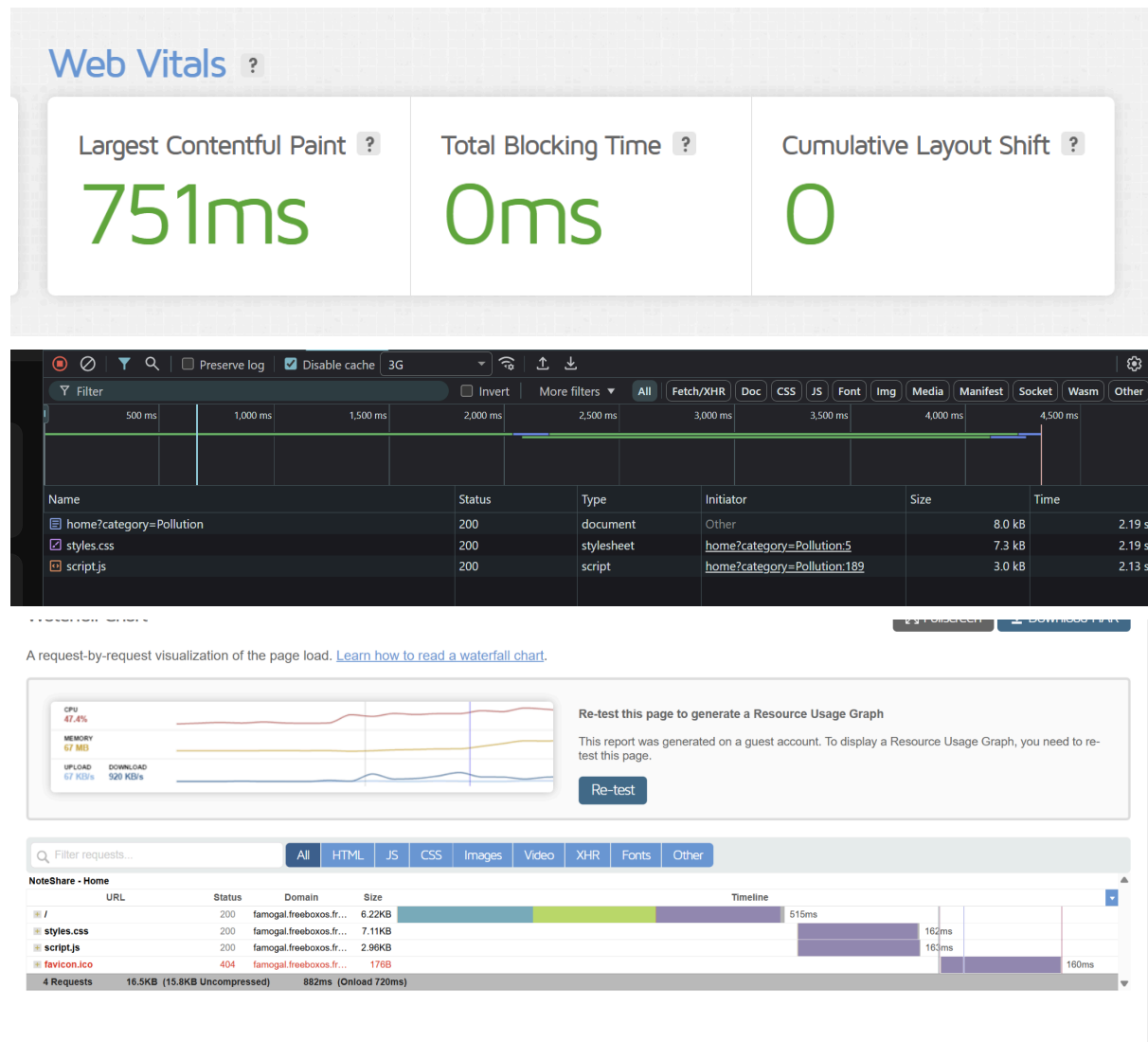
Indicator	Moodle	NoteShare (our website)	Gain
Homepage weight	3000KB	16KB	2984%
Number of HTTP requests	25	3	12
EcolIndex score	76/100	95/100	19pts
EcolIndex grade	B	A	B -> A
CO ₂ /visit	0.34g	0.01g	97%
Lighthouse Perf. score	51/100	100/100	49pts
FCP	0.8s	0.3s	62.5%

LCP	0.9s	0.3s	66%
-----	------	------	-----

VI. Testing and Validation

Create a valid user	✓
Modify an existing user	✓
Delete a user	✓
Login with valid credentials	✓
Login with wrong password	✓
Search study sheets by name / category	✓
Add a study sheet	✓
Display a study sheet	✓
Access protected page w/o login	✓





The page loads successfully, as all main resources return a 200 status. The only issue is a missing favicon.ico, which results in a 404 error but does not affect functionality. The load time is under one second, indicating good performance. No API requests are shown, so backend issues are not reflected here.

VII. Team Organisation and Collaboration

To properly work together and not stepping on each others foot when we are coding, we decided to divide the task into different issues on git to which we could assign ourselves to:

☐	☒	proper home page #11 · by Sandor12 was closed yesterday	
☐	☒	account page #10 · by Sandor12 was closed 2 weeks ago	
☐	☒	studysheet creation/import page #9 · by Sandor12 was closed 18 minutes ago	
☐	☒	Account page #8 · by Sandor12 was closed yesterday	 1
☐	☒	studysheet_formatting in the html #7 · by Sandor12 was closed 19 hours ago	
☐	☒	Searchbar algorithmic #6 · by Sandor12 was closed yesterday	
☐	☒	Proper main page #5 · by Sandor12 was closed 18 minutes ago	
☐	☒	General CSS #4 · by Sandor12 was closed yesterday	
☐	☒	Management through docker #3 · by Sandor12 was closed last week	 1
☐	☒	Database creation #2 · by Sandor12 was closed 16 minutes ago	
☐	☒	Login handler (page) #1 · by Sandor12 was closed yesterday	 1

This allowed us to easily see what each member was working on, which one was done and which one where available. From that, we would work on our tasks on a branch different from the main to avoid having to constantly fix merge problems. Once we finished, we merged the branch to the main and fixed the merge problems.

The contributions of each member are as follows

Feature	Person/group in charge
Login/sign up	Alexis Lafargue/ Alexis Delavis
Uploading content	Alexis Delavis
Delete content	Alexis Delavis

Alexis Delavis
Jude Aybalen
Thomas Bieth–Legout
Guillaume Bernard
Alexis Lafargue

Organized Storage	Thomas Bieth-Legout/Guillaume
Search	Guillaume Bernard/ Jude Aybalen
Comment section	Alexis Delavis/ Jude Aybalen
Deleting account and personal details	Thomas Bieth-Legout

VIII. Discussion and Conclusion

We encountered some problems especially when merging branches or with the development timeline, when some people worked faster than others and it made those faster people limited in their work. We also encountered some problems when trying to get a good security because some actions would have made the website much slower for exemple with the bcrypt number of rounds that we ended up setting at 12 but that we wanted to set at 20 which would have made the login and signup take approximately 10seconds. Finally we still have some accessibility problems because we didn't discover those soon enough for us to modify it.

Aside from this we haven't really made any tradeoffs between sobriety and sobriety thanks to the language we used and the fact that we're not using any tracker. As such our website achieves the highest performance on every ecological testing website we tried. The only one where we're not at the highest is because of accessibility that we didn't have the time to fix. The hosting on a Raspberry pi also made it really efficient because the cost of running it is much less than nearly any server you can access.

The main improvement we could add are the few accessibility features we're lacking and an ability to delete a sheet that we created. But we determined that it was not too necessary for the MVP when compared to the time of development it would have created. We could also make the commentary system look a bit better. Finally, the last thing which could be improved is to support more file format which is not really hard but some could lead to injections and we would want to avoid this as much as possible.

Finally, the decisions we took are all pretty good, but we didn't necessarily need docker as it just was an overhead in the testing part of the development instead of a real help. It was only really helping when deploying on the raspberry pi. For the rest, go lang still seems a really great choice as for the other choices of technology and the way of storing the sheets is really efficient there so it's easy to manage those. Finally, we shouldn't have put "editing a sheet" in our UML diagrams because we weren't able to do it correctly and properly.